EXAM NOTES

Inter-Process Communication with Pipes

P4 • ABC 12.3 • Beginner-Friendly Reference

1. BIG PICTURE — What Is IPC & Why Pipes?

When multiple processes need to talk to each other, they use Inter-Process Communication (IPC). Pipes are the simplest IPC mechanism on Linux/Unix.

IPC Methods (know all of these for MCQ)

Method	What It Is
Files	Processes read/write the same file on disk
Pipes	In-memory byte stream between related processes
Named Pipes (FIFO)	Pipe accessible by name; unrelated processes can use it
Sockets	Network-style communication; works across machines
Message Queues	Structured messages passed between processes
Shared Memory	Processes share a region of RAM directly
Semaphores/Signals	Synchronization primitives (control timing)

Shell Pipeline Reminder

In the shell, the | character creates a pipe between commands:
ls | tr a-z A-Z | wc
stdout of ls feeds into stdin of tr, then stdout of tr feeds into wc.
This is EXACTLY what pipes do in C — connect one process's output to another's input.

2. The pipe() System Call — Know This Cold

```
#include <unistd.h>
int pipe(int pipefd[2]);
             ^^^^^^^^^
             array of 2 ints filled by the OS
```

What pipe() Does

- Creates a one-directional (one-way) byte-stream buffer in the kernel
- Fills pipefd[0] — the READ end (think: 0 = input, like stdin)
- Fills pipefd[1] — the WRITE end (think: 1 = output, like stdout)
- Returns 0 on success, -1 on failure

Memory Trick: 0 reads, 1 writes

stdin is FD 0 → pipefd[0] is also 0 → both READ
stdout is FD 1 → pipefd[1] is also 1 → both WRITE
If you can't remember which end is which, think: 0 in, 1 out.

Standard File Descriptors — MUST Know

FD Number	Meaning
0	stdin — standard input
1	stdout — standard output
2	stderr — standard error
3, 4, ...	Assigned by OS to new files/pipes

3. The Fork + Pipe Pattern

The #1 pattern on exams: parent creates pipe, then forks. After fork, both parent and child have copies of both FDs. You must close the ends you don't use.

Step-by-Step: Child writes, Parent reads

Step 1 — Parent creates the pipe BEFORE forking

```
int p[2];
pipe(p);   // p[0] = read end,  p[1] = write end
           // Both FDs exist ONLY in the parent so far
```

Step 2 — Parent calls fork()

```
pid_t pid = fork();
// Now BOTH parent and child have p[0] and p[1] open.
// The pipe has 4 'references': parent-p[0], parent-p[1],
//                              child-p[0],  child-p[1]
```

⚠ Step 3 — CLOSE the ends you don't need (CRITICAL!)

Parent reads → Parent must close(p[1]) (write end)

Child writes → Child must close(p[0]) (read end)

Rule: Every process closes whichever end it does NOT use.
Failing to close causes hangs and EOF-detection bugs.

Step 4 — Communicate, then wait

Child: write(p[1], &data, sizeof(data));
exit(0); // child exits, closing its p[1]
Parent: read(p[0], &data, sizeof(data)); // blocking!
wait(NULL); // reap child

Complete Example — Child sends an int to Parent

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int p[2];
    pipe(p);            // create pipe

    pid_t pid = fork();

    if (pid == 0) {          // ── CHILD ──
        close(p[0]);         // child doesn't read
        int n = 42;
        write(p[1], &n, sizeof(int));
        exit(0);
    } else {                 // ── PARENT ──
        close(p[1]);         // parent doesn't write
        int result;
        read(p[0], &result, sizeof(int));  // blocks!
        printf("Received: %d\n", result);  // → 42
        wait(NULL);
    }
    return 0;
}
```

read() is BLOCKING — exam favourite!

When a process calls read() on a pipe and the pipe is empty:
→ The process SLEEPS (blocks) until data arrives.
→ It wakes up as soon as the writer writes something.
→ It gets EOF (read returns 0) only when ALL write ends are closed.
This is why closing unused write ends matters so much!

4. File Descriptor Tables — Reading the Diagrams

Every process has a File Descriptor Table (FDT). Each row is an FD number pointing to an entry in the OS Open File Table.

After pipe() — before fork()

```
Parent FDT
FD 0  →  stdin
FD 1  →  stdout
FD 2  →  stderr
FD 3  →  pipe READ  end   (= p[0])
FD 4  →  pipe WRITE end   (= p[1])
```

After fork() — both processes have all 4 FDs open

```
Parent FDT          Child FDT
FD 0  stdin         FD 0  stdin
FD 1  stdout        FD 1  stdout
FD 2  stderr        FD 2  stderr
FD 3  pipe-read     FD 3  pipe-read
FD 4  pipe-write    FD 4  pipe-write
```

After closing unused ends — CORRECT setup

```
   (Child writes, Parent reads)

Parent FDT          Child FDT
FD 0  stdin         FD 0  stdin
FD 1  stdout        FD 1  stdout
FD 2  stderr        FD 2  stderr
FD 3  pipe-read     FD 3  [CLOSED]
FD 4  [CLOSED]      FD 4  pipe-write
```

How to read FDT MCQs

1. Find which process is the writer and which is the reader.
2. The writer keeps p[1] open; everything else closed.
3. The reader keeps p[0] open; everything else closed.
4. An 'X' or blank in a slot = that FD is closed.

5. Two-Child Setup — Child1 → Child2

Goal: Child 1 writes to Child 2. Parent creates the pipe, forks twice, and closes BOTH ends (it neither reads nor writes).

Sequence

- Parent: pipe(p)
- Parent: fork() → creates Child 1
- **Child 1:** close(p[0]) — writer doesn't read
- Parent: fork() → creates Child 2
- **Child 2:** close(p[1]) — reader doesn't write
- **Parent:** close(p[0]); close(p[1]); — parent does nothing with pipe

```
int p[2];  pipe(p);

pid_t c1 = fork();
if (c1 == 0) {        // ── CHILD 1 (writer) ──
    close(p[0]);
    write(p[1], "hello", 5);
    exit(0);
}

pid_t c2 = fork();
if (c2 == 0) {        // ── CHILD 2 (reader) ──
    close(p[1]);
    char buf[10];
    read(p[0], buf, 5);
    printf("%s\n", buf);   // → hello
    exit(0);
}

// ── PARENT ──
close(p[0]);  close(p[1]);   // parent closes both
wait(NULL);   wait(NULL);    // reap both children
```

⚠ Exam Trap: When to close p[1] after Child 1 exits

After Child 1 exits it closes its p[1], BUT Child 2 also inherited p[1].
If you forget to have Child 2 close(p[1]), Child 2 will NEVER get EOF on read() because the write end is still 'alive' in Child 2 itself.
Rule: close every FD you don't use, in EVERY process.

6. dup2() — Redirecting stdin/stdout to a Pipe

When you run A | B in a shell, the shell uses dup2() to redirect A's stdout INTO the pipe write end, and B's stdin FROM the pipe read end.

dup2() Signature

```
#include <unistd.h>
int dup2(int oldfd, int newfd);
// Makes newfd point to the same file as oldfd.
// If newfd was open, it is closed first.
// Returns newfd on success, -1 on error.
```

Common dup2() Patterns

Goal	Code
Redirect stdout → pipe write end	`dup2(p[1], 1);`
Redirect stdin → pipe read end	`dup2(p[0], 0);`
Old way (same effect)	`close(1); dup(p[1]);`

Shell Pipeline (A | B) — Full Step-by-Step

Step 1: Shell S starts with FDs 0,1,2 only

S FDT: FD 0 (stdin), FD 1 (stdout), FD 2 (stderr)

Step 2: S calls pipe(p) → gets FDs 3 and 4

FD 3 = read end, FD 4 = write end

Step 3: Fork #1 creates process SA (for program A)

SA inherits all FDs: 0, 1, 2, 3, 4
SA: dup2(4, 1) → SA's stdout (FD 1) now points to pipe write end
SA: close(4); close(3) → clean up raw pipe FDs
SA: exec into program A

S: close(4) → S no longer needs the write end

Step 4: Fork #2 creates process SB (for program B)

SB inherits: 0, 1, 2, 3 (FD 4 already closed in S)

SB: dup2(3, 0) → SB's stdin (FD 0) now points to pipe read end
SB: close(3) → clean up raw pipe FD
SB: exec into program B

S: close(3) → S no longer needs the read end

Final State
A writes to FD 1 = pipe write end
B reads from FD 0 = pipe read end
S has NO pipe FDs open (clean!)
A and B run CONCURRENTLY

```
// Pseudocode for S implementing  A | B
pipe(p);                          // p[0]=3(read), p[1]=4(write)

pid_a = fork();
if (pid_a == 0) {                 // child for A
    dup2(p[1], 1);                // stdout → pipe write
    close(p[1]); close(p[0]);     // clean up
    exec("A", ...);               // run A
}
close(p[1]);                      // S doesn't write

pid_b = fork();
if (pid_b == 0) {                 // child for B
    dup2(p[0], 0);                // stdin → pipe read
    close(p[0]);                  // clean up
    exec("B", ...);               // run B
}
close(p[0]);                      // S doesn't read
waitpid(pid_a, ...);
waitpid(pid_b, ...);
```

7. FD Lifecycle — EOF, SIGPIPE & Cleanup

This section is exam gold — MCQ loves asking 'what happens when process X dies?'

When a process exits, ALL its open FDs are closed automatically.

Scenario	Result
`All writers close p[1]`	`Readers get EOF (read returns 0)`

A writer dies but ANOTHER process still has p[1] open	Reader BLOCKS (waits) — no EOF yet
All readers close p[0]	Next write() by writer gets SIGPIPE (error)
A reader dies but ANOTHER process still has p[0] open	Writer BLOCKS if pipe buffer is full

⚠ Most Common Exam Trap — Forgotten Parent FDs

If the parent creates a pipe but never closes it,
even after all children finish, the read end will NEVER get EOF
because the parent still has the write end open!

Always close pipe FDs in the parent if the parent doesn't use them.

PIPE_BUF and Atomicity

- write() is atomic (all-or-nothing) if you write ≤ PIPE_BUF bytes
- PIPE_BUF = 4096 bytes on Linux
- Writes larger than PIPE_BUF may be split and interleaved
- read() returns however many bytes are available (may be less than requested)
- Always check the return value of read()/write()!

8. Two-Way (Bidirectional) Communication

A single pipe is one-directional. For two-way communication between parent and child, you need TWO pipes.

Why Can't We Reuse One Pipe?

- If both processes try to read from the same pipe, they race
- No guarantee which process wins the race
- Solution: two pipes — one for each direction

Two-Pipe Setup (Parent sends n, Child returns sum)

```
//  parent_to_child[2]  →  parent writes, child reads
//  child_to_parent[2]  →  child writes, parent reads

int ptc[2], ctp[2];
pipe(ptc);  pipe(ctp);

pid_t pid = fork();
if (pid == 0) {          // ── CHILD ──
```

```
    close(ptc[1]);        // child doesn't write ptc
    close(ctp[0]);        // child doesn't read ctp
    int n;
    read(ptc[0], &n, sizeof(int));
    int sum = n*(n+1)/2;
    write(ctp[1], &sum, sizeof(int));
    exit(0);
} else {                  // —— PARENT ——
    close(ptc[0]);        // parent doesn't read ptc
    close(ctp[1]);        // parent doesn't write ctp
    int n = 10;
    write(ptc[1], &n, sizeof(int));
    int result;
    read(ctp[0], &result, sizeof(int));
    printf("Sum 1..%d = %d\n", n, result);  // → 55
    wait(NULL);
}
```

⚠ Deadlock Danger with Two Pipes

If BOTH parent and child call read() before either calls write(),
they will both block forever — a DEADLOCK.
Make sure one side writes first and the other side reads first.

9. Concurrent Matrix Example — Multiple Children

Classic exam problem: use N children to compute partial results, collect via one pipe.

Pattern: N children write to one pipe, parent reads N times

```
int pd[2];
pipe(pd);

for (int i = 0; i < N; i++) {
    if (fork() == 0) {            // each child
        int row_sum = add_vector(a[i]);
        write(pd[1], &row_sum, sizeof(int));
        return 0;                 // child exits
    }
}
// Parent should close pd[1] here!
// (once all children exit, parent gets EOF on read)

int total = 0;
for (int i = 0; i < N; i++) {
    int row_sum;
    read(pd[0], &row_sum, sizeof(int));
    total += row_sum;
```

```
}
printf("Total = %d\n", total);
```

Key Insight: Order of results is NOT guaranteed

Children run concurrently, so the first write() could be from any child.
The parent must read exactly N times (once per child), not assume order.
For ordered results, you'd need more complex synchronization.

10. Quick Reference — Exam Cheat Sheet

All System Calls You Need

Call	What It Does
`pipe(p)`	Create pipe; p[0]=read, p[1]=write
`fork()`	Duplicate process; returns 0 to child, PID to parent
`close(fd)`	Close a file descriptor
`read(fd,buf,n)`	Read up to n bytes; blocks if empty; 0 = EOF
`write(fd,buf,n)`	Write n bytes; blocks if pipe full
`dup2(old,new)`	Redirect new to point to old's file
`wait(NULL)`	Parent waits for ANY child
`waitpid(pid,...)`	Parent waits for a SPECIFIC child
`exec*(path,...)`	Replace process image; inherits open FDs
`exit(code)`	End process; all FDs closed automatically

Rules to Always Follow

- ALWAYS close the end of the pipe you don't use, in EVERY process
- ALWAYS close pipe FDs in the parent if parent doesn't use the pipe
- ALWAYS check return values of read()/write() — they may be less than requested
- ALWAYS create the pipe BEFORE fork() — not after
- For two-way comm: use TWO pipes
- After exec(), the new program inherits all open FDs

EOF Rules

- EOF on read() happens when ALL write ends (p[1]) are closed
- SIGPIPE on write() happens when ALL read ends (p[0]) are closed
- If even ONE extra write end is open, no EOF — reader blocks forever

dup2() Memory Aid

dup2(src, dst) — think: dst = src

dup2(p[1], 1) → FD 1 (stdout) now goes to the pipe write end
dup2(p[0], 0) → FD 0 (stdin) now comes from the pipe read end
After dup2(), you should close(src) — the raw pipe FD is no longer needed

Common MCQ Output Tracing Template

```
1. Trace which process runs which block (pid==0 is child, else parent)
```

```
2. Identify what is written into the pipe and by whom
3. Identify who reads and in what order (read BLOCKS if pipe empty)
4. If process exits before writing: remaining readers may block or get EOF
5. printf only flushes on newline (or fflush) — watch for buffering
```

11. Tricky MCQ Scenarios — Worked Examples

Scenario A: What is the output?

```
int p[2];  pipe(p);
if (fork() == 0) {
    close(p[0]);
    int x = 7;
    write(p[1], &x, sizeof(int));
    exit(0);
}
close(p[1]);
int y;
read(p[0], &y, sizeof(int));
printf("%d\n", y);
wait(NULL);
```

Answer: 7

Child writes 7 to p[1]. Parent reads from p[0] (blocks until child writes).
Parent prints 7. Child exits, parent's read() returns 4 bytes (sizeof int).

Scenario B: Will this hang?

```
int p[2];  pipe(p);
if (fork() == 0) {
    close(p[0]);
    write(p[1], "hi", 2);
    exit(0);
}
// Parent does NOT close(p[1]) !
char buf[10];
read(p[0], buf, 2);   // ← will this block forever?
printf("%s\n", buf);
wait(NULL);
```

⚠ Answer: read() returns, but it got lucky!

The child writes and exits (closing its p[1]).
BUT the parent still has p[1] open, so pipe's write end is still 'live'.
read() will return the data 'hi' from the buffer.

However, a SECOND read() would block forever (waiting for more writers).
Lesson: Always close(p[1]) in parent to ensure proper EOF.

Scenario C: After exec(), what FDs does the new program see?

```
int p[2];  pipe(p);
if (fork() == 0) {
    dup2(p[1], 1);      // stdout → write end
    close(p[0]);
    close(p[1]);        // close raw FD (stdout still connected)
    execlp("echo", "echo", "hello", NULL);
}
// Parent reads from p[0]...
```

Answer: echo writes to its stdout, which is now the pipe

dup2(p[1], 1) makes FD 1 point to the pipe write end.
After close(p[1]), FD 1 still works (pointing to pipe).
echo's output goes through FD 1 = into the pipe.
Parent reads 'hello\n' from p[0].

12. Glossary — Key Terms

Term	Definition
FD / File Descriptor	Integer handle a process uses to refer to an open file/pipe
FDT	File Descriptor Table — per-process array of FDs
pipe()	Syscall creating a unidirectional byte-stream buffer
fork()	Syscall duplicating a process; child inherits all FDs
exec()	Syscall replacing current process image; FDs inherited
dup2(old,new)	Makes FD 'new' point to same file as FD 'old'
read()	Reads bytes from FD; blocks if empty, returns 0 on EOF
write()	Writes bytes to FD; blocks if pipe buffer full
EOF	End-of-file; pipe signals this when all write ends closed
SIGPIPE	Signal sent to writer when all read ends are closed
IPC	Inter-Process Communication — any way processes exchange data

Atomic write	Write ≤ PIPE_BUF bytes guaranteed not to be split
PIPE_BUF	4096 bytes on Linux — max atomic write size
Blocking	Process sleeps until operation can complete
Deadlock	Two+ processes wait for each other forever

Good luck on your exam! Remember: pipe first, fork second, close what you don't use.